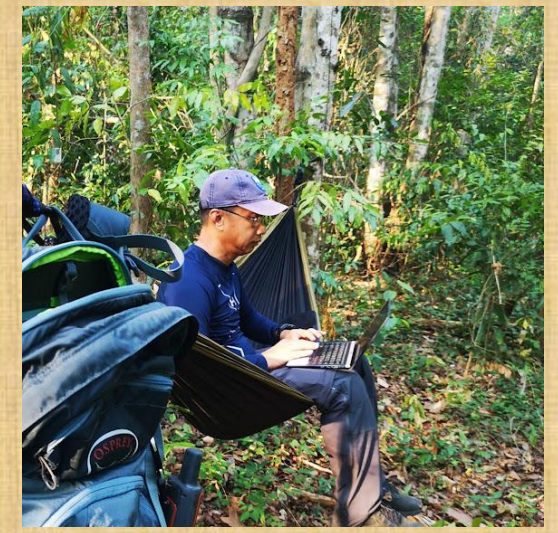
ITCS214 : DATA STRUCTURES & ALGORITHM ANALYSIS



Lecture 4 · Stacks, Queues & Deques



Restricted-access collections: one end, the other end, or both

Asst. Prof. Dr. Ananta Srisuphab
ananta.sri@mahidol.ac.th
2026/1

Mahidol University
Faculty of Information and Communication Technology

What you will be able to do after today

Learning outcomes

- Specify stack, queue, and deque ADTs independently of any implementation
- State representation invariants for array-based and linked implementations
- Trace push/pop, offer/poll, and both-end operations, including wrap-around
- Apply stacks to delimiter matching, infix→postfix, and postfix evaluation
- Apply queues to BFS and scheduling; apply deques to palindromes and sliding windows
- Choose exception-throwing vs special-value Java methods deliberately
- Distinguish worst-case from amortized O(1) for resizable arrays

Today's route

1. Compare Stack · Queue · Deque
2. Stack ADT, arrays, links, Java
3. Stack applications & traces
4. Queue ADT & circular arrays
5. Queue applications, PriorityQueue
6. Deque ADT, both-end algorithms
7. Complexity, testing, decision guide

An ADT is a contract; an implementation is a choice

Abstract data type (the what)

- A set of operations with defined behavior
- Rules for boundary cases: empty, full, invalid
- No commitment to memory layout or language
- Example: “pop() removes and returns the most recently pushed element; on empty, it throws.”

Implementation (the how)

- A concrete representation: array, linked nodes, …
- A representation invariant that must hold between operations
- Concrete costs: time per operation, space overhead
- Example: “elements[0..size-1] hold the items; size == 0 means empty.”

Design rule: specify behavior first, then pick a representation that meets the cost targets.

Neither the stack nor the queue ADT requires a fixed capacity — bounded capacity is an implementation decision.

Three ADTs answer one question: who leaves next?

STACK

LIFO — last in, first out

Insert and remove at one end only (the top). The most recent item leaves first.

Undo, call stack, DFS

QUEUE

FIFO — first in, first out

Insert at the rear, remove at the front. The earliest item leaves first.

Print jobs, BFS, buffering

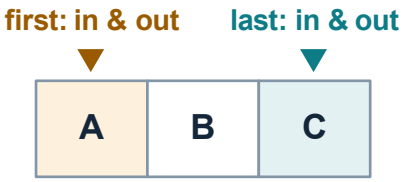
DEQUE

Double-ended queue

Insert and remove at either end. Generalizes both the stack and the queue.

Sliding window, work stealing





One color convention for the whole lecture: amber marks the top/front; teal marks the rear/back.

PART 1 · ONE ACCESSIBLE END

Stacks

push · pop · peek — the most recent item always leaves first

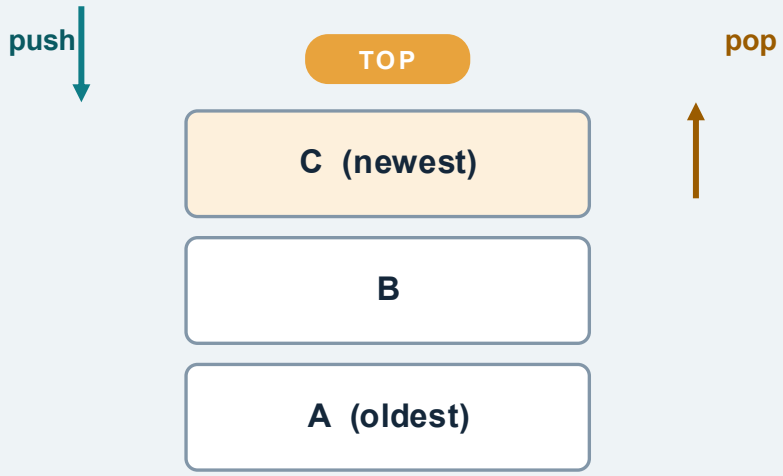
The stack contract: only the top is visible

Operations

`push(x)`	adds x on top of the stack
`pop()`	removes and returns the top element
`peek()`	returns the top element without removing it
`isEmpty()`	reports whether the stack has no elements
`size()`	returns the number of elements (optional)

Every core operation is O(1) — no traversal, ever.

Only one door



Insertion and removal share the same end. Elements below the top are unreachable until everything above them is popped.

Predict the state after every operation

Operation sequence

```
push(10);
push(20);
pop();      // returns ?
push(30);
peek();     // returns ?
pop();      // returns ?
pop();      // returns ?
pop();      // then what?
```

Write the stack contents after each line.

State after each step (right side = top)

`push(10)` [10]

`push(20)` [10, 20]

`pop()` → [10]

`push(30)` [10, 30]

`peek()` → 30 [10, 30]

`pop()` → [10]

`pop()` → [0]

`pop()` → StackUnderFlowError

Boundary behavior is part of the specification

Underflow

pop() or peek() on an empty stack has no valid element to return. The ADT must say what happens.

Overflow

A fixed-capacity implementation can fill. A resizable one grows instead — but memory is still finite.

Empty-operation policies

- Throw NoSuchElementException
- return Optional.empty()
- return a documented special value
- require an isEmpty() pre-check.

Never invent a magic return value

- **Returning -1 on empty is only safe if -1 can never be a stored element — rarely true for a generic Stack<E>.**
- Java's special-value queue methods return null on empty — legal only because null elements are banned (ArrayDeque rejects them).
- Also part of the contract: the ADT itself never requires a fixed capacity — boundedness is an implementation property.

Array stack invariant: items[0..size–1] are the stack

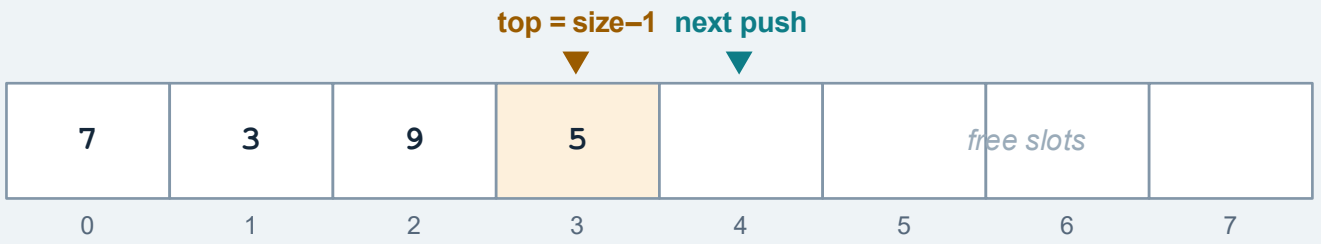
Representation invariant

- **items[0 .. size–1] hold the elements, bottom to top**
- size == 0 ⇔ the stack is empty
- top element (if any) lives at items[size – 1]
- slots items[size ..] are free (and hold no stale references)

Cost profile

- push / pop / peek: O(1) (push amortized if resizable)
- Space: one contiguous array — excellent cache locality
- Fixed-capacity variant: push on full → overflow policy

size = 4, capacity = 8



Grows to the right;
top is the last occupied index.

push and pop in full — including the boundary lines

```
public class ArrayStack<E> {
  private E[] items;
  private int size = 0;

  @SuppressWarnings("unchecked")
  public ArrayStack(int capacity) {
    items = (E[]) new Object[capacity];
  }

  public void push(E e) {
    if (size == items.length)
      resize(2 * items.length);  // grow
    items[size++] = e;
  }
}
```

```
public E pop() {
  if (isEmpty())
    throw new NoSuchElementException();
  E top = items[--size];
  items[size] = null; // no loitering
  return top;
}

public E peek() {
  if (isEmpty())
    throw new NoSuchElementException();
  return items[size - 1];
}

public boolean isEmpty() {
  return size == 0;
}
```

items[size] = null lets the GC reclaim the popped object — skipping it causes memory loitering.

Doubling makes push O(1) amortized, not O(1) always

Growth policy: double when full

```
private void resize(int newCap) {
  items = Arrays.copyOf(items, newCap);
}                          // O(size) copy
```

One expensive push pays for the many cheap pushes after it.

Total copy work for n pushes

- Copies happen at sizes 1, 2, 4, …, n/2
- **1 + 2 + 4 + … + n/2 < n**
- n pushes ⇒ < 2n element writes total
- **⇒ O(1) amortized per push; O(n) worst-case for one push**

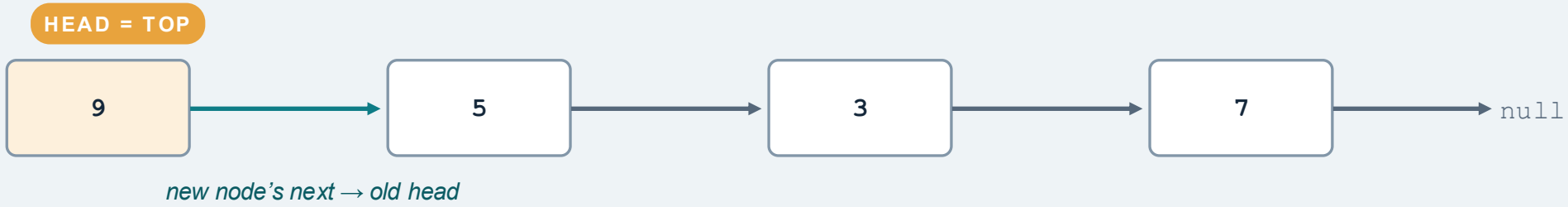
Say it precisely

Worst-case: the cost of the single most expensive operation. Amortized: total cost of a sequence divided by its length.

“push is O(1) amortized and O(n) worst-case for a doubling array; O(1) worst-case for a linked stack.”

A linked stack pushes and pops at the head

head is the top — push(9) in two link updates




```
private static final class Node<E> {
  E item;  Node<E> next;
  Node(E item, Node<E> next)
    { this.item = item; this.next = next;
}
}
private Node<E> head = null;  // top
private int size = 0;
```

```
public void push(E e) {
  head = new Node<>(e, head);
  size++;
}
public E pop() {
  if (head == null) throw
    new NoSuchElementException();
  E top = head.item;
  head = head.next;  size--;
  return top;
}
```

O(1) worst-case for every operation · no capacity limit · cost: one node object + link per element.

In modern Java, a stack is a Deque used from one end

```
// Program to the interface
Deque<Integer> stack = new ArrayDeque<>();

stack.push(10);          // addFirst
stack.push(20);
int x   = stack.pop(); // 20 (removeFirst)
int top = stack.peek();// 10 (peekFirst)

// Empty stack: pop/peek throw
// NoSuchElementException — or use
// pollFirst()/peekFirst() → null
```

Why not java.util.Stack?

- Legacy class (Java 1.0) that extends Vector
- Inherits index-based insert/remove — clients can violate LIFO
- Synchronized on every call: overhead without real thread-safety guarantees for compound actions
- Javadoc: “a more complete and consistent set of LIFO stack operations is provided by the Deque interface.”

Default choice: ArrayDeque<E> — resizable circular array, O(1) amortized at both ends, faster than Stack and LinkedList in practice.

Pick one method family (push/pop/peek here) and use it consistently within a class.

Delimiter matching: the stack holds unresolved openers

Algorithm

1. **Scan**

   read one character at a time

2. **Opener ([{**

   push it

3. **Closer)] }**

   stack empty? → fail; else pop and check the pair matches

4. **End of input**

   balanced only if the stack is empty

Trace: { [( )] } ( )

char	action	stack (top at right)
{	push	{
[	push	{ [
(	push	{ [(
)	pop (— match ✓	{ [
]	pop [— match ✓	{
}	pop { — match ✓	(empty)
(	push	(
)	pop (— match ✓	**(empty) → balanced ✓**

Complete delimiter matcher — every boundary is explicit

```
static boolean isBalanced(String s) {
  Deque<Character> stack = new ArrayDeque<>();
  for (char c : s.toCharArray()) {
    if (c == '(' || c == '[' || c == '{') {
      stack.push(c);                    // opener
    } else if (c == ')' || c == ']' || c == '}') {
      if (stack.isEmpty()) return false;   // nothing open
      char open = stack.pop();
      if (!matches(open, c)) return false; // e.g. ( vs ]
    }                                   // other chars: ignore
  }
  return stack.isEmpty(); // leftover openers ⇒ unbalanced
}
static boolean matches(char open, char close) {
  return open == '(' && close == ')'
      || open == '[' && close == ']'
      || open == '{' && close == '}';
}
```

Boundary checklist

")(" → false

closer meets empty stack

"(]" → false

popped opener wrong type

"((" → false

opener never closed

"" → true

vacuously balanced

Time O(n), space O(n) worst case — the stack never holds more than the current nesting depth.

Infix → postfix: pop while the top has priority

Rules when a token arrives

- **Operand → append it directly to the output**
- **Operator op → pop to output while the top is an operator with higher precedence — or equal precedence and op is left-associative — then push op**
- "(" → push it (a barrier no operator pops past)
- ")" → pop to output until "("; discard both parentheses
- End of input → pop everything to output

Why equal precedence pops

8 − 3 − 2 is (8 − 3) − 2 = 3

correct output: 8 3 − 2 −

If equal precedence did NOT pop:

8 3 2 − − ⇒ 8 − (3 − 2) = 7 ✗

Mnemonic: the stack holds operators that are still waiting for their right operand; anything that must happen first gets flushed out.

Historical note: the original lecture stated this comparison backwards — verify against 8 − 3 − 2 whenever unsure.

Worked trace: 3 * (4 + 5) − 6

token	action	operator stack (top right)	output so far
3			3
*		*	
(		*(	
4			3 4
+		*(+	
5			3 4 5
)		*	
−			3 4 5 + * 6 -
6			
end			

Postfix evaluation: pop right first, then left

Algorithm

Operand → push it

Operator →

right = pop() (pushed last)

left = pop() (pushed first)

push(left op right)

End → exactly one value remains: the result

Fewer than two values at an operator, or more than one at the end ⇒ malformed expression

```
Deque<Double> stack = new ArrayDeque<>();
for (String tok : expr.split("\\s+")) {
  if (isNumber(tok)) {
    stack.push(Double.parseDouble(tok));
  } else {
    double right = stack.pop(); // last in
    double left  = stack.pop();
    stack.push(apply(tok, left, right));
  }
}
return stack.pop(); // sole remaining value
```

Order test: 6 3 / must give 2, not 0.5. Subtraction and division expose the bug; + and * hide it.

Worked trace: 3 4 5 + * 6 1 2 + / −

token	action	stack after (top right)
3 4 5		3 9
+		27
*		27
6 1 2		6 3
+		2
/		27 - 2
−		25

Stacks appear wherever work must resume later

Undo / redo history

Each edit pushes its inverse (or a snapshot). Undo pops the latest change; redo is a second stack, cleared by any new edit.

The call stack

Every method call pushes an activation record — locals, parameters, return address. Returning pops it. Recursion depth = stack depth.

Backtracking & DFS

Push each decision point; on a dead end, pop back to the most recent choice and try another branch. Mazes, puzzles, parsers.

Expression processing

Compilers use the two algorithms you just saw — conversion and evaluation — plus delimiter checks, all stack-driven.

Pattern to recognize: "suspend now, resume the most recent thing first" ⇒ use a stack.

Two-minute exercise before we move on

Predict, then compare with a neighbor

```
Deque<Integer> s = new
ArrayDeque<>();
s.push(1);  s.push(2);  s.push(3);
System.out.println(s.pop());
System.out.println(s.pop());
s.push(4);
System.out.println(s.pop());
```

Hint: write the stack contents after every line; the right end is the top.

Q1. What is printed, and what remains on the stack?

Q2. True or false: peek() decreases size() by one.

Q3. One more pop() after this code — what happens, and which policy decides?

Answers are in the speaker notes — commit to a prediction first.

PART 2 · ARRIVAL ORDER

Queues

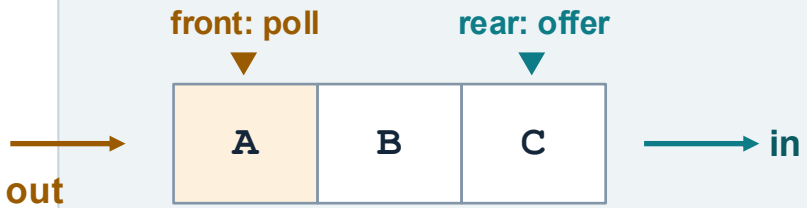
offer · poll · peek — the earliest item always leaves first

The queue contract: enqueue at the rear, dequeue at the front

Operations (classical name → Java name)

`enqueue → offer(x)`	adds x at the rear
`dequeue → poll()`	removes and returns the front element
`front → peek()`	returns the front element without removing it
`isEmpty() / size()`	no arguments — isEmpty(int i) is a signature error

Two doors, one direction



A entered first, so A leaves first — arrival order is preserved.

State trace (front at left)

`offer(10) → [10]`

`offer(20) → [10, 20]`

`poll() returns 10 → [20]`

`peek() returns 20 → [20]`

Java queues offer two method families — pick one deliberately

operation	exception family (throws)	special-value family (signals in the return)
insert	`add(e)` — `IllegalStateException` if capacity-constrained and full	`offer(e)` — returns `false` instead
remove	`remove()` — `NoSuchElementException` if empty	`poll()` — returns `null` instead
examine	`element()` — `NoSuchElementException` if empty	`peek()` — returns `null` instead

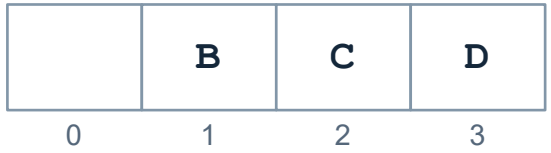
How to choose

Empty means a bug in my logic → exception family (fail fast). **Empty is a normal state to handle → special-value family.**

Consequence of poll() → null: queues that support this family must forbid null elements — ArrayDeque throws NullPointerException on offer(null).

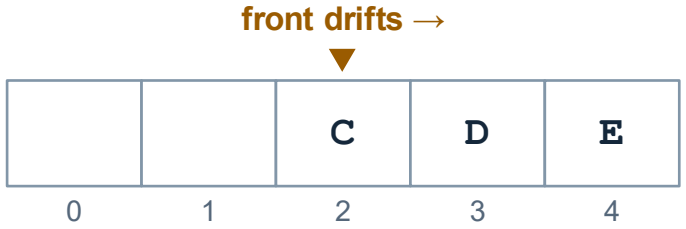
Why the obvious array queue is too slow

Attempt A — shift left on poll



- poll() removes index 0, then shifts every element one slot left
- **poll becomes O(n) — unacceptable for a core operation**

Attempt B — advance front, never reuse

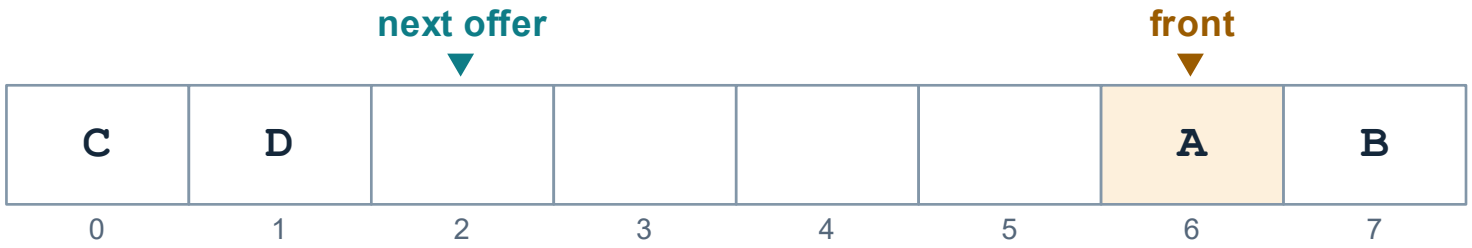


- poll() just moves front right — O(1) ✓
- **but vacated slots on the left are never reused: space grows with operations, not with contents**

Goal: O(1) offer and poll AND space proportional to the number of stored elements. Fix: let indices wrap around — a circular array.

A circular array: physical indices vs logical order

Physical array — capacity 8, front = 6, size = 4



next offer index = (front + size) % capacity = (6 + 4) % 8 = 2 — the rear wraps past the end and continues at 0

Logical order (what clients see)



front … rear

Representation invariant

- **element i sits at (front + i) % capacity**
- empty ⇔ size == 0 full ⇔ size == capacity

Circular queue in code: modulo does the wrapping

```
public class CircularQueue<E> {
  private E[] items;
  private int front = 0, size = 0;

  public boolean offer(E e) {
    if (size == items.length)
      resize(2 * items.length);
    int rear = (front + size)
               % items.length;
    items[rear] = e;
    size++;
    return true;
  }
```

```
  public E poll() {
    if (size == 0) return null;
    E value = items[front];
    items[front] = null;   // GC
    front = (front + 1)
            % items.length;
    size--;
    return value;
  }

  public E peek() {
    return size == 0 ? null
         : items[front];
  }
}
```

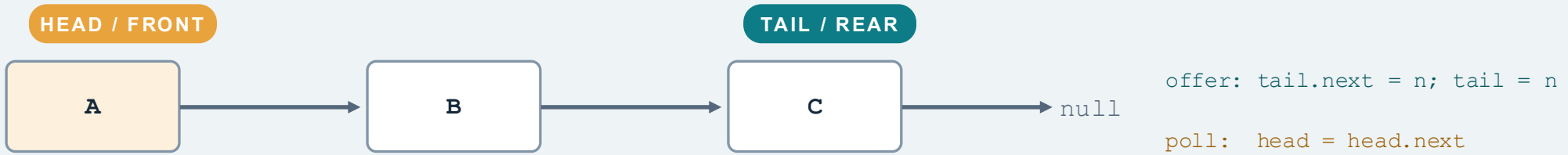
offer: O(1) amortized (resizing) · poll/peek: O(1) · no shifting, ever. Resize copies in logical order so front returns to 0.

Predict the wrap-around: capacity 4, no resizing

operation	physical array [0..3]	front	size	logical contents
`offer A,B,C,D`	`[A, B, C, D]`	`0`	`4`	`A B C D  (full)`
`poll() → A`				
`poll() → B`				
`offer(50) → index ?`	capacity 4 size 2 front 2 = 0			
`offer(60) → index ?`	1			

A linked queue needs both a head and a tail

head = front (poll here) · tail = rear (offer here)



Empty-state invariant

- **Empty ⇔ head == null AND tail == null**
- First offer: set both head and tail to the new node
- Poll of the final node: also set tail = null — forgetting this leaves tail pointing at a removed node

```
public E poll() {
  if (head == null) return null;
  E v = head.item;
  head = head.next;
  if (head == null) tail = null;
  size--;  return v;
}
```

offer / poll / peek: O(1) worst-case · no capacity limit · one node + link of overhead per element.

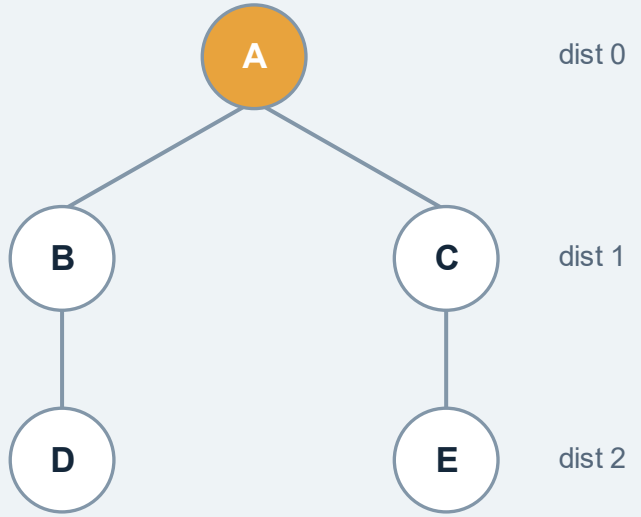
Breadth-first search: the queue enforces level order

Algorithm

```
Queue<V> q = new ArrayDeque<>();
mark(start);
q.offer(start);
while (!q.isEmpty()) {
  V v = q.poll();        // earliest
  process(v);
  for (V w : neighbors(v))
    if (!marked(w)) {
      mark(w);           // on enqueue!
      q.offer(w);
    }
}
```

Queue → shortest paths by edge count. Stack → depth-first.

Level-order guarantee



Visit order from A: A · B · C · D · E

All of level k is processed before any of level k+1.

Queues model fairness and decoupling

CPU & task scheduling

Round-robin schedulers rotate ready tasks through a FIFO queue; each gets a fair time slice in arrival order.

Buffering & messaging

Producers enqueue work; consumers dequeue at their own pace. Keyboard buffers, I/O, message brokers — the queue absorbs speed mismatches.

Event processing

GUI and server event loops poll events in the order they occurred, preserving causality.

Simulation

Discrete-event models represent waiting customers, jobs, or packets as FIFO queues to measure waiting times and throughput.

Pattern to recognize: “serve in arrival order” or “decouple producer from consumer” ⇒ use a queue.

A priority queue is a different ADT — not a FIFO with extras

FIFO queue

- **Removal rule: earliest arrival leaves first**
- Time is the only key
- offer / poll / peek: O(1)
- Natural fit: fairness, buffering, BFS

Priority queue

- **Removal rule: smallest (or largest) key leaves first**
- java.util.PriorityQueue: binary heap in an array — no front/rear pointers
- offer / poll: O(log n) · peek: O(1)
- Natural fit: Dijkstra, schedulers with deadlines, top-k

Two facts exams love

Iterating a PriorityQueue is NOT sorted — only repeated poll() is.

Ties are broken arbitrarily — it is not FIFO among equal priorities.

PART 3 · BOTH ENDS

Deques

addFirst · addLast · removeFirst · removeLast — the client chooses the end

Deque = double-ended queue: six operations per end

Front (first) operations

exception family	special-value family
`addFirst(e)`	`offerFirst(e)`
insert at the front	
`removeFirst()`	`pollFirst()`
remove and return the front	
`getFirst()`	`peekFirst()`
examine the front	

Back (last) operations

exception family	special-value family
`addLast(e)`	`offerLast(e)`
insert at the back	
`removeLast()`	`pollLast()`
remove and return the back	
`getLast()`	`peekLast()`
examine the back	

Same boundary discipline at both ends: remove/get throw NoSuchElementException on empty; poll/peek return null instead.

“Deque” (the noun, pronounced “deck”) names the structure; “dequeue” (the verb) means removing from a queue.

One Deque interface supports all three access policies

As a stack — LIFO

one end only (the front)

```
Deque<Integer> st =
  new ArrayDeque<>();
st.push(1);
st.push(2);
st.pop();  // 2
st.peek(); // 1

// push = addFirst
// pop  = removeFirst
```

As a FIFO queue

in at the back, out at the front

```
Deque<String> q =
  new ArrayDeque<>();
q.offerLast("A");
q.offerLast("B");
q.pollFirst(); // A
q.peekFirst(); // B

// same as Queue's
// offer / poll
```

As a true deque

either end, per operation

```
Deque<String> d =
  new ArrayDeque<>();
d.addFirst("mid");
d.addFirst("front");
d.addLast("back");
// front, mid, back
d.pollLast();  // back
d.pollFirst(); // front
```

Pick one discipline per object and keep to it — the interface will not stop a client from mixing ends by accident.

ArrayDeque: the practical default — know its two caveats

Why it is the default

- Resizable circular array — the structure from this lecture, productionized
- O(1) amortized insertion and O(1) removal at both ends
- Better cache behavior than LinkedList; no per-element node objects
- Serves as stack, queue, and deque with one class

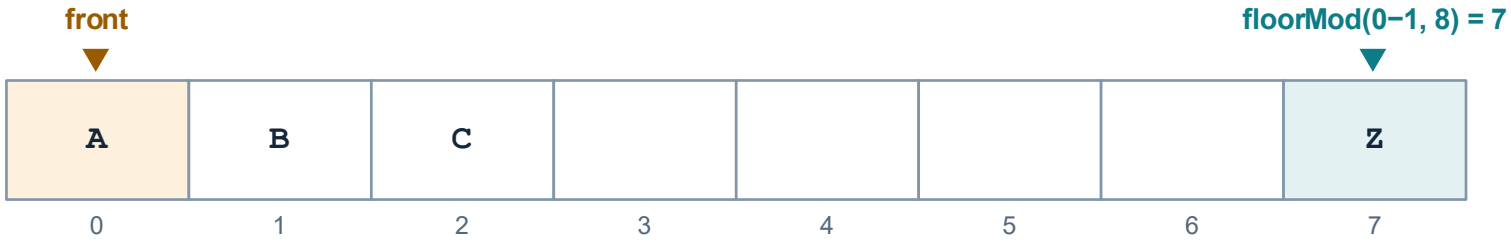
Caveats

- **null elements are forbidden — offer(null) throws NullPointerException**
- Reason: null is the “empty” signal of pollFirst()/peekFirst(); storing it would be ambiguous
- **Not thread-safe — no internal synchronization**
- Concurrent use → ConcurrentLinkedDeque or a BlockingDeque implementation

Single-threaded stack or queue work in modern Java ⇒ reach for ArrayDeque first.

The circular array extends naturally to both ends

front = 0, size = 3 — where does addFirst(Z) write?



addFirst wraps backward — the deque grows leftward past index 0

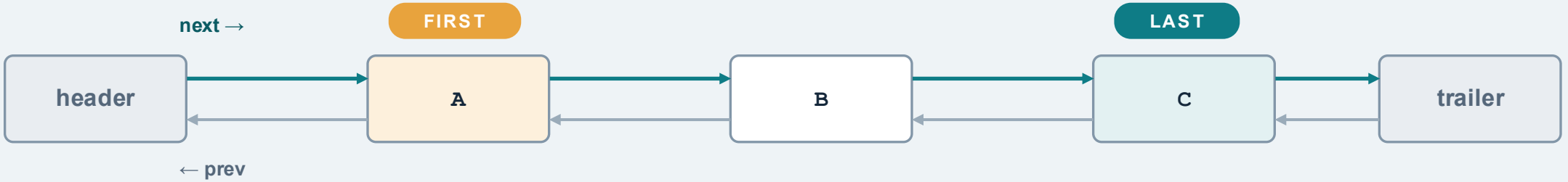
```
public void addFirst(E e) {
  if (size == items.length) resize(...);
  front = Math.floorMod(front - 1,
                        items.length);
  items[front] = e;  size++;
}
```

```
public E removeLast() {
  if (size == 0) throw new NoSuchElementException();
  int last = (front + size - 1) % items.length;
  E v = items[last];  items[last] = null;
  size--;  return v;
}
```

(front − 1) % capacity can be negative in Java — Math.floorMod is the safe backward wrap. All end operations: O(1).

A doubly linked deque: sentinels remove the special cases

header and trailer are permanent dummies — real nodes live between them



Invariant with sentinels

- **first element = header.next · last = trailer.prev**
- **empty ⇔ header.next == trailer**
- insertBetween(x, a, b) implements all four end operations

Trade-off vs circular array

- O(1) worst-case (no resize pauses) at both ends
- Costs two links + node header per element; weaker locality
- No random access in either design — not a List

Palindrome checking uses both ends at once

Algorithm

```
static boolean isPalindrome(String s) {
  Deque<Character> d = new ArrayDeque<>();
  for (char c : s.toCharArray())
    d.addLast(c);
  while (d.size() > 1) {
    if (d.removeFirst()
        != d.removeLast())
      return false;
  }
  return true; // 0 or 1 char left
}
```

O(n) time · both-end removal is the whole trick.

Trace: RACECAR

R A C E C A R — R = R ✓

A C E C A — A = A ✓

C E C — C = C ✓

E — **size ≤ 1 → true ✓**

Counterexample STACK: first pair S vs K fails immediately.

Sliding-window maximum: a monotonic deque in O(n)

Per new element a[i]

1. **Expire**

   pollFirst() while the front index has left the window (≤ i − k)

2. **Dominate**

   pollLast() while a[back] ≤ a[i] — smaller values can never be a future max

3. **Append**

   offerLast(i)

4. **Report**

   a[peekFirst()] is the window maximum

Trace: a = [1, 3, −1, −3, 5], k = 3

i : a[i]	deque (indices : values)	max
0 : 1	[0 : 1]	–
1 : 3	3 evicts 1 → [1 : 3]	–
2 : −1	[1, 2 : 3, −1]	3
3 : −3	[1, 2, 3 : 3, −1, −3]	3
4 : 5	5 evicts all → [4 : 5]	5

Values in the deque stay strictly decreasing — the front is always the max.

Each index enters and leaves at most once ⇒ O(n) total (amortized O(1) per step).

Where deques earn their keep in real systems

Work-stealing schedulers

Each worker owns a deque: it pushes/pops its own tasks at one end (LIFO, cache-hot); idle workers steal the oldest task from the other end. Used by Java's ForkJoinPool.

Bounded histories

Browser history or undo with a size cap: add the newest entry at one end, evict the oldest from the other when full.

Sliding-window analytics

Monotonic deques answer window max/min queries in O(n) — rate limiting, streaming statistics, stock spans.

Palindromes & symmetry

Any check that consumes a sequence from both ends toward the middle.

Pattern to recognize: two parties (or two rules) each need their own end of one sequence ⇒ use a deque.

SYNTHESIS

Representation scorecard: every end operation is O(1)

	resizable circular array	singly linked (head/tail)	doubly linked + sentinels	Java library
Stack push/pop	O(1) amortized / O(1)	O(1) at head	O(1)	ArrayDeque
Queue offer/poll	O(1) amortized / O(1)	O(1) tail / head	O(1)	ArrayDeque
Deque both ends	O(1) amortized	— (no O(1) removeLast)	O(1) worst-case	ArrayDeque
Resize pause?	yes — occasional O(n)	never	never	array-based: yes
Space per element	1 slot (≤ 2× over-alloc)	node + 1 link	node + 2 links	—
Cache locality	excellent	poor	poor	excellent

Amortized O(1) ≠ O(1) worst-case: a doubling array occasionally pays O(n) on one operation; linked structures never do.

SYNTHESIS

Decision guide: name the removal rule, then pick the ADT

“The most recent item leaves first”	**Stack**	undo, call stack, DFS, delimiter matching
“The earliest arrival leaves first”	**Queue**	scheduling, buffering, BFS, simulation
“Either end, chosen per operation”	**Deque**	sliding window, work stealing, bounded history
“The smallest / largest key leaves first”	**PriorityQueue**	Dijkstra, deadlines, top-k selection

Then choose the implementation: learning → build it · single-threaded production → ArrayDeque / PriorityQueue · concurrent → java.util.concurrent deques and queues.

Test the boundaries first — that is where invariants break

Six classic bugs

- Off-by-one at the top: peek reads items[size], not items[size − 1]
- Missing % capacity: index runs off the array after enough wraps
- Negative wrap: (front − 1) % capacity instead of floorMod
- Stale tail: last node polled but tail not reset to null
- Reversed operands: left = pop() before right = pop()
- Loitering: vacated slots keep references alive

Boundary test plan

- empty → one element → empty again
- fill to capacity − 1 → full → one more (policy fires?)
- force wrap-around, then drain completely
- add at both ends, remove at both ends (deque)
- repeated grow (and shrink) cycles
- empty-operation calls: does the documented policy fire?

Rule of thumb: if a test suite never reaches full, empty-after-use, and wrapped states, it has not tested the structure.